Chapter 7 - Graphs

7.1 Introduction

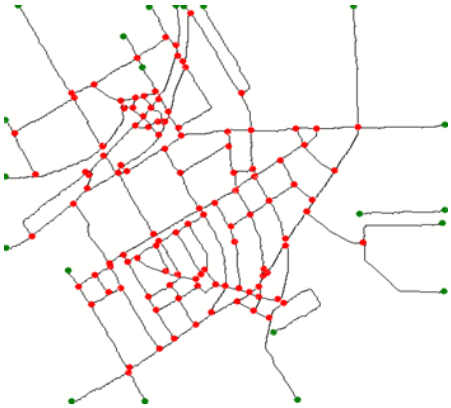


Figure 1: A graph for a road network[3]

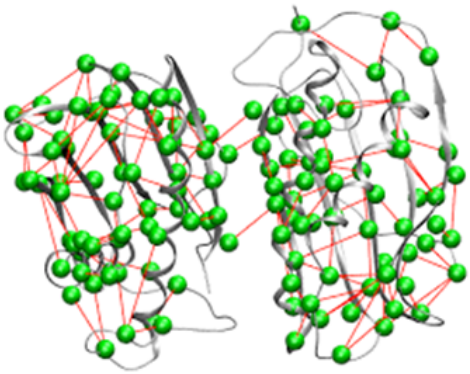


Figure 2: a graph showing a Protein Structure[4]

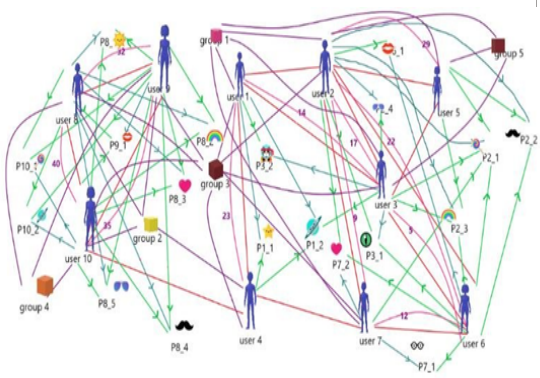


Figure 3: A Facebook network[6]

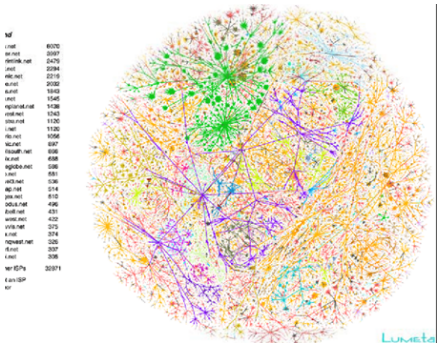


Figure 4: graph visualizing the internet[5]

So far, we have learned about data structures that can store linear sequences (e.g., arrays, lists, stack, queues) and data structures that can represent non-linear hierarchical relationships (i.e., different types of trees). What can be the most generic data structures that can represent arbitrary relationships among entities without any ordering restrictions? The answer is graphs. The graph is the most versatile data structure that can represent any relationship network among a collection of objects. For example, we can use graphs to represent road networks, the

[3] Image source: https://www.researchgate.net/figure/An-example-of-road-network-extraction-and-graph-representation_fig12_279263325

[4] Image source: https://benthamopen.com/contents/supplementary-material/TOBIOIJ-5-53_SD1.pdf

[5] Image source: https://www.researchgate.net/figure/2-A-graph-visualisation-of-the-topology-of-network-connections-of-the-core-of-the_fig2_239550496

[6] Image source: https://www.researchgate.net/publication/340347681_A_Study_on_Graph_Theory_Properties_of_On-line_Social_Networks

internet, structure of molecules and proteins, social networks, evolutionary relationships among species, geographic regions, and so on.

In fact, a single graph can capture multiple types of entities and relationships. Whenever we have a collection of entities and need to model their interactions and relations as a network, we can use graphs. Therefore, being able to store and use graphs in our programs is absolutely essential. Graphs are so important in computer science that graph theory and graph algorithms are considered core computer science courses and many books have been written on them. Even drawing a graph for visualization and human analysis is a vast topic taught as a separate course in many universities! Here we will only learn the basics related to graphs: the terminologies we have to understand, common graph data structure representations, and how to traverse a graph.

7.2 Graph Terminologies

A graph G is represented as a set of vertices (or nodes) V that represent the entities/objects/concepts and a set of edges (or links) E that represent relationship/interaction among those vertices. So, we typically write a graph G as G = (V, E). If a and b are two vertices in V and if there is an edge between them in the graph, we commonly write the edge as (a, b). When there is an edge (a,b) in E, we say vertex a and b are adjacent or neighbors. Note that the relationship a graph captures may be directional or undirected. In the former case we have a directed graph, in the latter case, an undirected graph. Below are examples of a directed and an undirected graph. We typically draw edges in an undirected graph as lines/curves and edges in a directed graph as arrows.

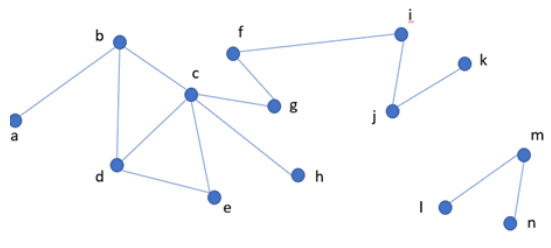


Figure 5: an undirected graph

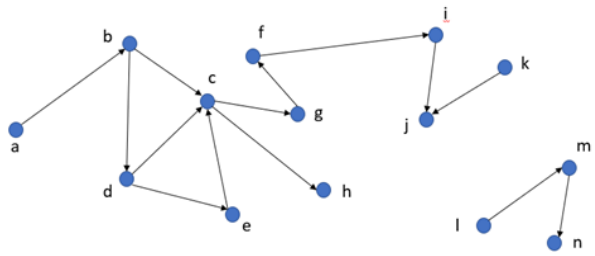


Figure 6: a directed graph

Notice that in a undirected graph having an edge (a,b) in E means the same thing as having the edge (b,a). However, in a directed graph (a,b) means an edge from a to b, while (b,a) means an edge from b to a. We can have none, either, or both in a directed graph. If we have both the edges, then we have to draw two arrows (one from a to b and another from b to a) when drawing the graph.

The **degree** of a vertex in an undirected graph is the number of edges the vertex has. For example, in Figure 5, degree of vertex d is 3. In a directed graph, a vertex has both in-degree

and out-degree. The former represents the number of edges originating from other vertices and ending at that vertex, the latter represents the opposite. So, the in-degree of vertex d in Figure 6 is 1 and out-degree is 2. A directed edge (a,b) in E is an outgoing edge of a and an incoming edge of b.

A **path** between two vertices x and y in a graph is a sequence of edges of the form ($x = u_0$, u_1),(u_1, u_2),(u_2,u_3),…,(u_{n-1}, u_n=y). For example, the sequences of edges (a,b),(b,c),(c,e),(e,d) forms a path between vertex a and d in the graph of Figure 5. The length of the path is the number of edges in it. Note that, there may be zero, one, or more paths between any two vertices in a graph. When there is no path between two vertices u, v in a graph then the graph is called **disconnected**. Then we also say that u and v are unreachable from each other. The subset of vertices that are mutually reachable from one another along with the various edges form a **component** of a disconnected graph. Hence, there are two components in the graph of Figure 5.

Note that in the directed graph of Figure 6, there is no path (a,b),(b,c),(c,e),(e,d) as the edges are directional. However, vertex d is still reachable from a as we have the path (a,b),(b,d). Notice that, a is not reachable from d in this case, this is again because the edges are directional. The **distance** of a vertex v from another vertex u in a graph is the number of edges in the shortest path from u to v.

A **cycle** in a graph is a sequence of edges that starts and ends at the same vertex. For example in Figure 5, (b,d),(d,e),(e,c),(c,b) edge sequence form a cycle. In a directed graph, the cycles are also directional. Hence, there is no directed cycle in the graph of Figure 6.

In both directed and undirected cases, the vertices and/or the edges of the graph can have weights assigned to them. Then we call the graph a weighted directed/undirected graph. The weight assigned to a vertex is used to represent the importance of the entity/object/concept it represents. The weight assigned to an edge typically represents the cost or strength of the relationship among the vertices it connects. For example, in a road network graph, the vertices are road junctions and the edges are road segments. Then the weight of a junction can be the maximum traffic capacity at that junction and the weight of the edge between two junctions can be their distance, aka, the length of the road. Below are two examples of weighted graphs.

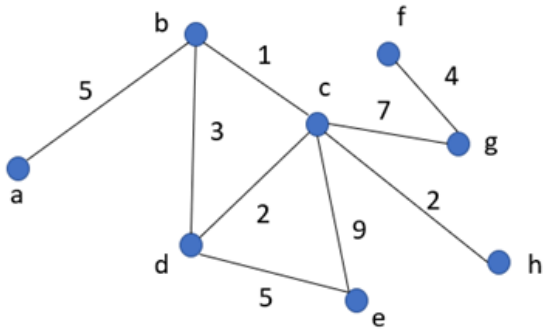


Figure 7: an edge weighted undirected graph

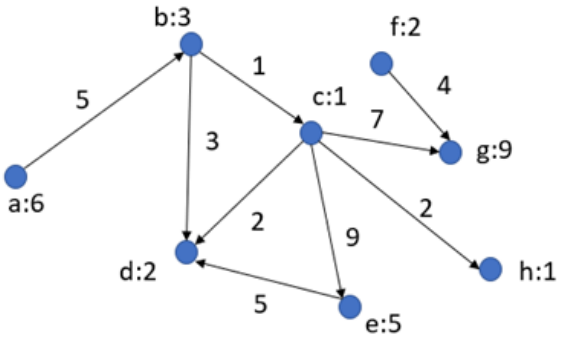


Figure 8: directed graph having both vertex and edge weights

We say a graph is sparse when the number of edges is too low compared to the number of vertices. We call it a dense graph when the number of edges is high. This distinction is important when we choose among various data structure representations of a graph.

There are many more definitions related to graphs. However, for our purpose, knowing up to this much is sufficient. Given an undirected, unweighted graph is the most common case, in the rest of this chapter we will use the term graph to mean an undirected, unweighted graph by default. When we refer to the other cases, we mention them explicitly.

7.3 Graph Representations

Unlike a tree, a graph generally does not have a root vertex/node that we can use to recursively discover all other vertices. The reason for that is simple. As the graph may be disconnected, it may be impossible to explore the entire graph if we are given only a single starting vertex. Hence, in any typical graph representation we have all the vertices and edges given. The most common representations of a graph are the following:

1. Adjacency List Representation
2. Adjacency matrix representation, and
3. Incidence matrix representation

7.3.1 Adjacency List Representation

In this representation, the vertices of the graph are represented by the indices of an array. Each entry of the array is a linked list. The elements of the list contain indices of the vertices that are adjacent to the vertex owning the index.

Let us consider the graph of Figure 5, and assume vertices a to n are assigned successive increasing indices starting from 0. Then the adjacency list representation of the graph is as follows:

a 0	b 1	c 2	d 3	e 4	f 5	g 6	h 7	I 8	j 9	k 10	l 11	m 12	n 13
1	0 2 3	1 3 4 6 7	1 2 4	2 3	6 8	2 5	2	5 9	8 10	9	12	11 13	12

Table 1: adjacency list representation of graph in Figure 5

It is quite easy to represent a directed graph using this format also. Everything remains the same, we just include the edge in the entry for the vertex where the edge starts from – not where it ends. Therefore, the adjacency list representation of the directed graph of Figure 6 is as follows:

a 0	b 1	c 2	d 3	e 4	f 5	g 6	h 7	I 8	j 9	K 10	l 11	m 12	n 13
1	2 3	6 7	2 4	2	8	5		9		9	12	13	

Table 2: Adjacency list representation of the graph in Figure 6

When we have a weighted graph to represent then this plain array of list of numbers is not sufficient. Let us consider the most generic case, where both the vertices and the edges can have weights. We now have to construct an Edge class to hold all information about an edge and a separate array holding the node weights. Then the Edge class should look as follows:

```
Class Edge {
        int src
        int dest
        Int weight
}
```

Here in the properties of the Edge class, ep1 and ep2 are the indices of the vertices that an edge connects. With this modification, we can represent the weighted directed graph of Figure 8 as follows.

Node Weight Array	a	b	c	d	e	f	g	h
	0	1	2	3	4	5	6	7
	6	3	1	2	5	2	9	1

Adjacency List	a	b	c	d	e	f	g	h
	0	1	2	3	4	5	6	7
	<0,1,5>	<1,2,1> <1,3,3>	<2,3,2> <2,4,9> <2,6,7> <2,7,2>		<4,3,5>	<5,6,4>		

Table 3: Adjacency list representation of the weighted directed graph of Figure 8

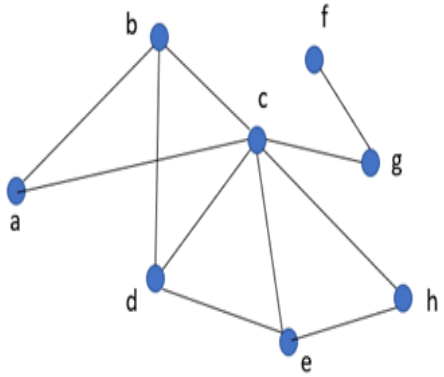
In the above, the tuple <x,y,z> stands for <ep1,ep2,weight> of an edge. Notice that in the directed weighted graph case, the edge class can record only the destination vertex of an edge; the source is not needed as it is the same as vertex owning the index of the array holding the list of edges. However, the way we defined the edge class, we can represent both directed and undirected weighted graphs using that class.

Finally, note that the vertices and edge can have other properties along with their weights in graph representations of real-world scenarios. In that case, we can define a Node class for the vertices and have an array of nodes and include more properties related to the edges in the Edge class. One can further combine the two arrays by having a node class holding the list of edges along with a vertex's other properties.

Space Complexity: if a graph has N vertices and M edges, then the adjacency matrix representation needs an array of size N and the total number of nodes in various lists in different array indices will be total 2M (in case of undirected graph) or M (in case of a directed graph). Hence the space requirement for the adjacency list representation is proportional to N + M. In the asymptotic notation, we can write the space complexity as $O(N+M)$.

7.3.2 Adjacency Matrix Representation

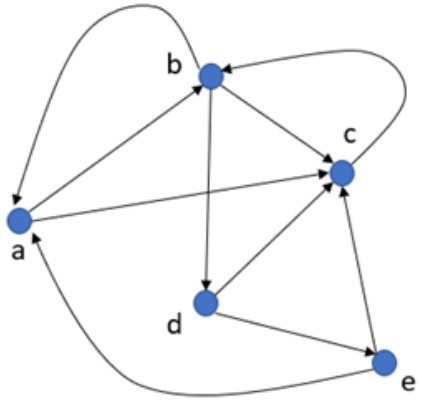
The adjacency matrix representation of a graph with N vertices uses an N×N matrix. As in the case of previous representation, vertices are assigned increasing indices starting from 0. The i^{th} row and column of the matrix are for the i^{th} vertex of the graph. The entry at i^{th} row and j^{th} column of the matrix is 1 if there is an edge between the corresponding vertices of the graph. Below is the drawing and adjacency matrix of a graph as an example.



	0	1	2	3	4	5	6	7
a 0	0	1	1	0	0	0	0	0
b 1	1	0	1	1	0	0	0	0
c 2	1	1	0	1	1	0	1	1
d 3	0	1	1	0	1	0	0	0
e 4	0	0	1	1	0	0	0	1
f 5	0	0	0	0	0	0	1	0
g 6	0	0	1	0	0	1	0	0
h 7	0	0	1	0	1	0	0	0

Table 4: an adjacency matrix representation of an undirected unweighted graph

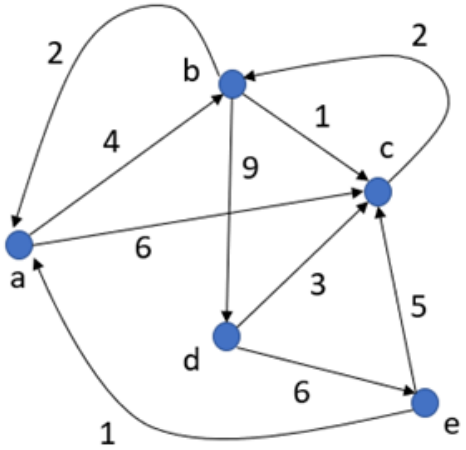
Notice that the matrix is symmetric. That is if we call the matrix A then each A[i][j] entry is the same as A[j][i] entry for all i,j < N. The reason for this is simple, as the edges are undirected, each edge occurs twice in the matrix. The adjacency matrix of a directed graph is not symmetric, unless of course when for each (a,b) edge of the graph there is also a (b,a) edge from vertex b to vertex a. Below is an adjacency matrix representation of a small directed graph.



	0	1	2	3	4
a 0	0	1	1	0	0
b 1	1	0	1	1	0
c 2	0	1	0	0	0
d 3	0	0	1	0	1
e 4	1	0	1	0	0

Table 5: an adjacency matrix representation of a directed graph

If a graph has only weight on its edges, then we can represent it using the adjacency matrix just as easily in both directed and undirected cases. If there is an edge (a,b) with weight w in the graph and the indices of a and b are i and j in the matrix respectively, then we simply write w in the <i,j> cell of the adjacency matrix, instead of writing 1. If the edge is undirected then we do the same in <j,i> cell also. Below is an adjacency representation when the earlier directed graph is edge-weighted (i.e., only the edges have weights).



	0	1	2	3	4
a 0	0	4	6	0	0
b 1	2	0	1	9	0
c 2	0	2	0	0	0
d 3	0	0	3	0	6
e 4	1	0	5	0	0

Table 6: adjacency matrix representation of a weighted directed graph

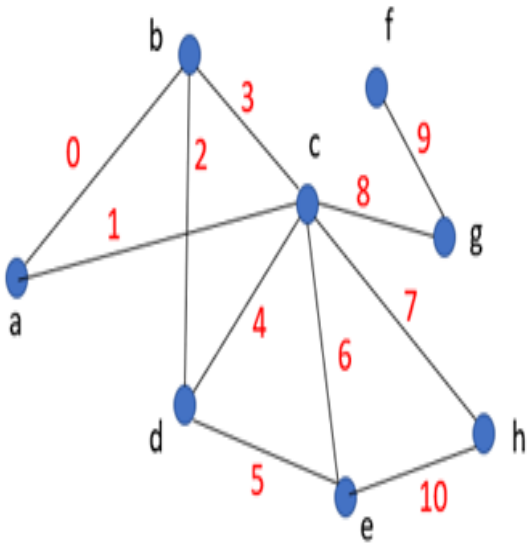
When both vertices and the edges of a graph have weights then we need a node weight array as in Table 3 to represents the weights of the vertices along with the adjacency matrix that captures the edge weights. Write the adjacency matrix representation of the graph of Figure 8 as an exercise of this case.

Space Complexity: for a graph with N vertices and M edges, the adjacency matrix representation will take N×N entries, that is, N^2 entries. Consequently, the asymptotic space complexity in this representation is $O(N^2)$. Apparently, the space cost of adjacency matrix representation is significantly higher than that of adjacency list representation. Then why do we use this second representation? The answer is many computations are easier in the matrix representation than in the list representation. We trade space with time when using any representation.

7.3.3 Incidence Matrix Representation

The final commonly used representation for graphs is the incidence matrix representation. This representation is particularly useful and popularized by electrical circuit analysis where edges represent wires with resistance/inductance/capacitance and the vertices represents junctions where the edges connect and where voltage being applied. You would be surprised to know that this representation is more than 150 years old!

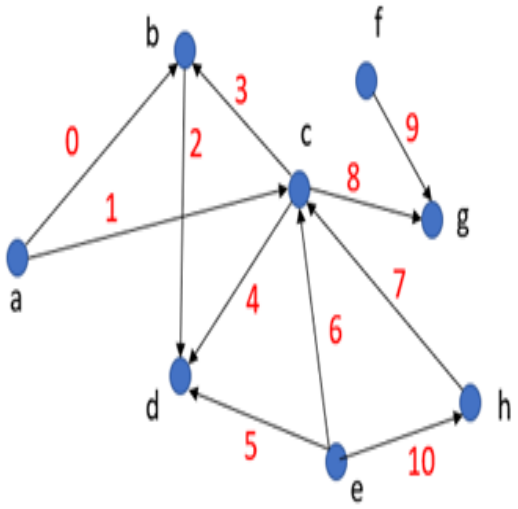
In the incidence matrix representation, the edges and vertices both are numbered. Thus, if there are M edges and N vertices in the graph, edges are numbered from 0 to M – 1 and vertices are from 0 to N – 1. Then a M×N matrix is used where the rows are the edges and the columns are the vertices. We write a 1 in the two columns of a row to indicate that these are the two vertices connected by the edge owning that row. The remaining entries of the row are filled with zeros. Below is an example (here the indices of the edges are shown in red color).



	a:0	b:1	c:2	d:3	e:4	f:5	g:6	h:7
0	1	1	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0
2	0	1	0	1	0	0	0	0
3	0	1	1	0	0	0	0	0
4	0	0	1	1	0	0	0	0
5	0	0	0	1	1	0	0	0
6	0	0	1	0	1	0	0	0
7	0	0	1	0	0	0	0	1
8	0	0	1	0	0	0	1	0
9	0	0	0	0	0	1	1	0
10	0	0	0	0	1	0	0	1

Table 7: The incidence matrix representation of an undirected graph

When we have a directed graph, we put a 1 in the source of the edge and a -1 in the destination (or vice versa). Below is an example.



	a:0	b:1	c:2	d:3	e:4	f:5	g:6	h:7
0	1	-1	0	0	0	0	0	0
1	1	0	-1	0	0	0	0	0
2	0	1	0	-1	0	0	0	0
3	0	-1	1	0	0	0	0	0
4	0	0	1	-1	0	0	0	0
5	0	0	0	-1	1	0	0	0
6	0	0	-1	0	1	0	0	0
7	0	0	-1	0	0	0	0	1
8	0	0	1	0	0	0	-1	0
9	0	0	0	0	0	1	-1	0
10	0	0	0	0	1	0	0	-1

Table 8: incidence matrix representation of a directed unweighted graph

You probably already understand how to represent an edge-weighted graph in this representation. Instead of writing 1 and -1, we have to put the positive and negative weight of the edge in the proper columns in a row.

Space Complexity: for a graph with N vertices and M edges, the incidence matrix representation will require M×N entries. Consequently, the asymptotic space complexity in this representation is $O(MN)$. When the underlying graph is sparse, the incidence matrix representation may be cheaper than the adjacency matrix representation or similar in cost. When the opposite is true, the adjacency matrix is better. Electrical circuits are typically sparse graphs, that is why incidence matrix representation is quite popular in circuit analysis. However, there are algorithms that specifically need the incidence matrix representation for computation efficiency.

	Pros	Cons
Adjacency Matrix	● Simple and intuitive structure ● Constant-time edge lookup **O(1)** ● Efficient for dense graphs ● Easy to implement using arrays	● High space usage **O(V^2)** even for sparse graphs ● Inefficient for iterating over neighbors (**O(V)** per vertex) ● Adding/removing vertices is costly
Adjacency List	● Space efficient **O(V + E)** ● Ideal for sparse graphs. ● Efficient neighbor traversal- Flexible structure (easy to add/remove edges) ● Commonly used in real-world applications	● Edge lookup slower (**O(degree of vertex)**) ● Slightly more complex implementation ● Less efficient for dense graphs compared to matrix
Incidence Matrix	● Clearly represents edge–vertex relationships ● Useful in algebraic graph theory and network flow problems ● Can represent multigraphs easily	● Space inefficient **O(V × E)** ● Edge lookup less direct than adjacency matrix- More complex to interpret and use in practice ● Rarely used in standard implementations

Table 9: Pros and Cons of Adjacency Matrix, Adjacency List and Incidence matrix representations.

When to use which representation:

- Use Adjacency Matrix when the graph is dense and fast edge lookup is required.
- Use Adjacency List when the graph is sparse and memory efficiency matters.
- Use Incidence Matrix mainly in theoretical or specialized applications rather than general-purpose implementations.

Practice Problems:

1. Given an unweighted graph in the adjacency matrix representation, calculates the number of edges the graph has.
2. Given a directed unweighted graph in the adjacency matrix representation, generate the adjacency list representation of the graph.
3. Solve the following problems for all three types of representation of the input graph.
    a. Given an undirected, unweighted graph as input, write a function to find the vertex with maximum degree and return the degree of that vertex.
    b. Given an undirected, edge-weighted graph as input, write a function to find the vertex whose sum of edge weights is maximum.
    c. Solve Problem #a and #b for directed, edge-weighted graph considering only outgoing edges.

7.4 Graph Traversals

To solve many interesting graph problems, one must first understand how to traverse a graph. In graph traversal, we start from an arbitrary vertex and follow its edges to reach neighboring vertices. We then continue the same process for those neighbors. At first glance, this may seem similar to traversing a tree starting from a root node and recursively visiting its children. However, graph traversal introduces **two important complications**.

First, unlike a tree, a graph is not necessarily a hierarchical data structure. A graph may contain cycles, meaning that during traversal we may eventually return to a previously visited vertex. If we use a straightforward recursive approach without any precautions, the traversal can enter an infinite loop by repeatedly moving around the same cycle.

Second, a graph may be disconnected. In such a case, starting the traversal from a particular vertex allows us to visit only the vertices belonging to the same connected component as that starting vertex. The remaining components of the graph will remain unvisited.

The two most common graph traversal algorithms are **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**.

DFS is often implemented recursively, much like tree traversal, because recursion naturally behaves like a stack (and uses the Call Stack). DFS can also be implemented explicitly using a stack instead of recursion. Although both recursive DFS and stack-based DFS follow the same depth-first principle, the exact traversal order may differ slightly depending on how neighbors are inserted into and removed from the stack. BFS, on the other hand, is typically implemented iteratively using a queue.

Unlike trees, graph traversal requires us to keep track of visited vertices in order to avoid revisiting the same vertices indefinitely in the presence of cycles. In general, graph traversal algorithms rely on auxiliary data structures such as queues or Stack (even if we use recursion, we're technically using the Call Stack), together with a list or set (basically a collection) that keeps track of visited vertices.

The general iterative approach works as follows. First, we insert the starting vertex into the stack or queue and also mark it as visited by adding it to the visited list. We then repeatedly execute a loop until the stack or queue becomes empty. In each iteration, we remove a vertex from the stack or queue using either the pop operation (for a stack) or the dequeue operation (for a queue). After processing that vertex, we examine all of its neighboring vertices. Any neighbor that has not already been visited is inserted into the stack or queue and simultaneously added to the visited list.

This procedure guarantees that all vertices reachable from the starting vertex are eventually visited. If any vertices are unvisited that means those vertices were unreachable from that specific starting point. This happens when a portion of the graph is disconnected or it is a directed graph where one or more vertices have no outgoing edges. In such cases, to visit all vertices an additional step is necessary. After the initial traversal process ends, we check whether there are still vertices that have not been visited. If such a vertex exists, we select it as a new starting vertex and repeat the traversal process. By doing so, we can ensure that every vertex in the graph is eventually visited.

Below are pseudo-codes of the graph traversal process.

BFS Traversal

```
visited = empty set

//#This process ensures that no vertex is unvisited

for each vertex v in graph:
    if v is not visited:

        enqueue v //#Here, v is the starting vertex
        mark v visited

        while queue is not empty:

            current = dequeue
            process(current)

            for each neighbor n of current:
                if n is not visited:
                    enqueue n
                    mark n visited
```

DFS Traversal (using Stack)

```
visited = empty set

//#This function ensures that no vertex is unvisited

for each vertex v in graph:
    if v is not visited:

        push v into stack //#Here, v is the starting vertex
        mark v visited

        while stack is not empty:

            current = pop stack
            process(current)

            for each neighbor n of current:
                if n is not visited:
                    push n into stack
                    mark n visited
```

DFS Traversal (using Recursion)

```
//#This function ensures that no vertex is unvisited
DFS_Traversal(Graph G)

    create empty Visited set

    for each vertex v in G
        if v is not visited
            DFS(G, v, Visited)

//#This function takes a starting point and only visits the reachable vertices
DFS(Graph G, Vertex v, Visited)

    mark v as visited
    process(v)

    for each neighbor u of v in G
        if u is not visited
            DFS(G, u, Visited)
```

7.4.1 Breadth First and Depth First Traversal Simulations

We have two very different graph exploration patterns depending on whether we used a stack (or recursion) or a queue in the aforementioned traversal algorithms. First, let us examine the behavior of the **Breadth-First graph traversal** for a small graph using a queue to store the vertices to be visited next. For simplicity's sake, we use an example of a connected graph.

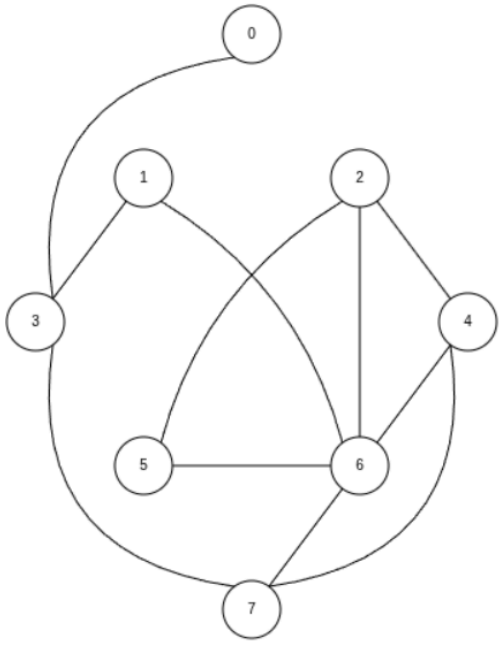
7.4.1.1 BFS SIMULATION:

We'll be simulating **BFS using 3 extra data structures**:

- **A Queue:** to store the neighbors of the currently processing vertex
- **A Visited Array:** A boolean array to keep track of vertices which are visited
    - By default all the vertices will be unvisited (false)
    - As we visit each vertex, we'll make them true.
- **A Parent Array:** to keep track of the edges/paths the algorithm has discovered.
    - By default the values are going to be null.

BFS Traversal on an Undirected Unweighted Graph

Step 1: Enqueuing the starting vertex into the Queue



We're starting from the '3' vertex. So it was enqueued into the Queue and marked as visited.

Queue: 3

Visited Array

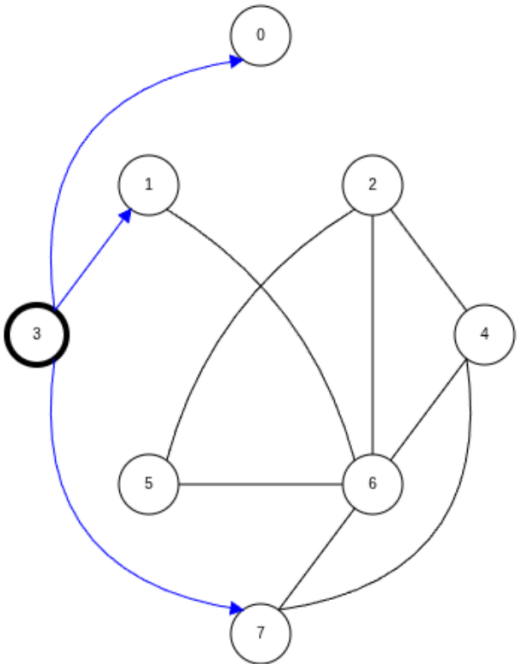
0	1	2	3	4	5	6	7
F	F	F	**T**	F	F	F	F

Parent Array

0	1	2	3	4	5	6	7
N	N	N	N	N	N	N	N

Step 2: dequeuing vertex 3 and processing its neighbors

`Traversal Sequence: 3`



We deque vertex 3 and then add the neighbors of vertex 3 into the queue. The order in which we add them is irrelevant. We must mark them as visited as well.

`Queue: 0 1 7`

Visited Array

0	1	2	3	4	5	6	7
T	**T**	F	**T**	F	F	F	**T**

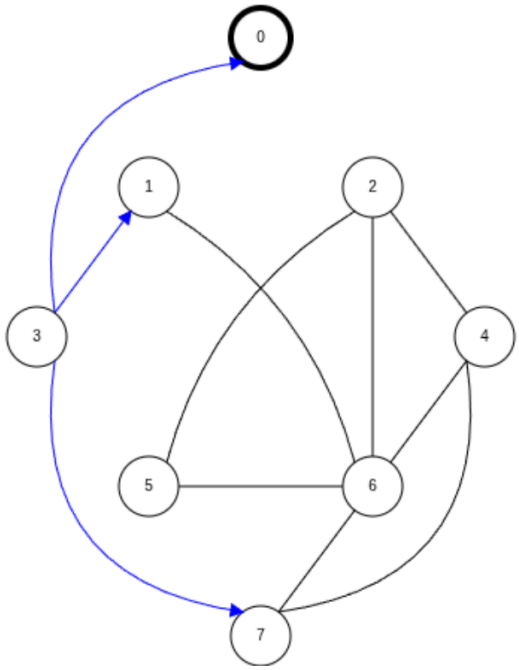
Parent Array

0	1	2	3	4	5	6	7
3	**3**	N	N	N	N	N	**3**

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

Step 3: dequeuing 0 and processing its neighbors

`Traversal Sequence: 3 0`



We deque vertex 0 and nothing really happens because ***it has no unvisited neighbors***.

`Queue: 1 7`

Visited Array

0	1	2	3	4	5	6	7
T	**T**	F	**T**	F	F	F	**T**

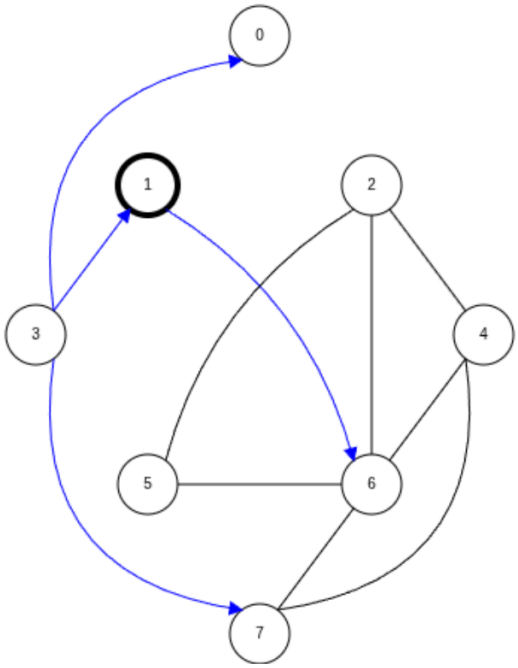
Parent Array

0	1	2	3	4	5	6	7
3	**3**	N	N	N	N	N	**3**

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

Step 4: dequeuing 1 and processing its neighbors

Traversal Sequence: 3 0 1



We deque vertex 1 and add its unvisited neighbor vertex 6 to the queue and mark 6 as visited.

Queue: 7 6

Visited Array

0	1	2	3	4	5	6	7
T	T	F	T	F	F	T	T

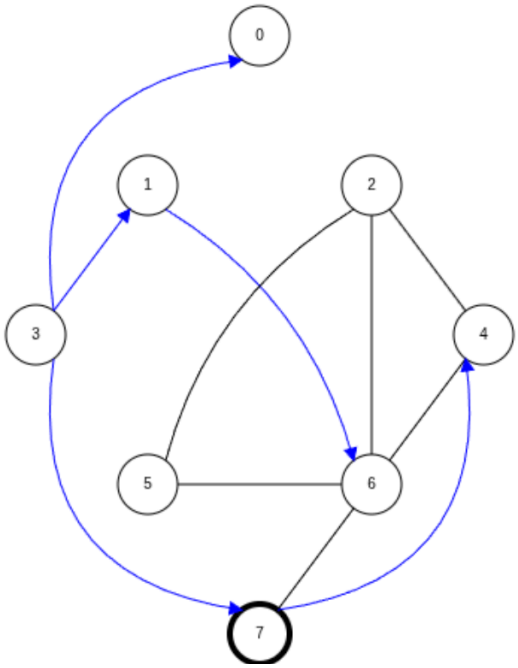
Parent Array

0	1	2	3	4	5	6	7
3	3	N	N	N	N	1	3

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

Step 5: dequeuing 7 and processing its neighbors

Traversal Sequence: 3 0 1 7



We deque vertex 7 and add its unvisited neighbor vertex 4 to the queue and mark 4 as visited. Vertex 6 was ignored since it was already visited.

Queue: 6 4

Visited Array

0	1	2	3	4	5	6	7
T	T	F	T	T	F	T	T

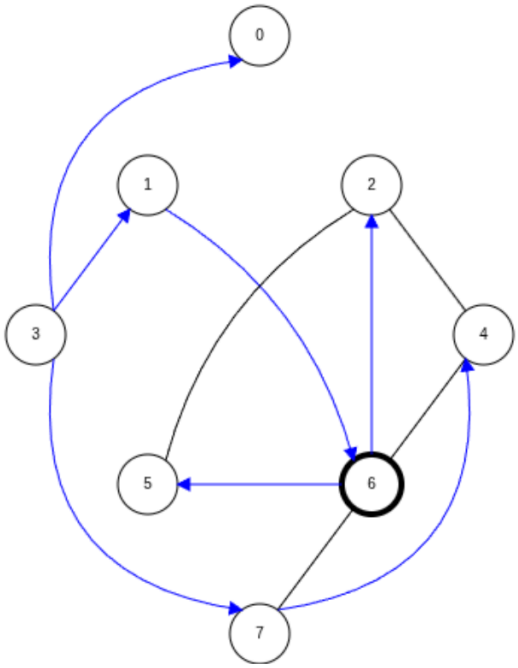
Parent Array

0	1	2	3	4	5	6	7
3	3	N	N	7	N	1	3

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

Step 6: dequeuing 6 and processing its neighbors

Traversal Sequence: 3 0 1 7 6



We deque vertex 6 and add its unvisited neighbor vertex 2 and 5 to the queue and mark 2&5 as visited. Vertex 4&7 were ignored as they were already visited.

Queue: 4 2 5

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	T	T

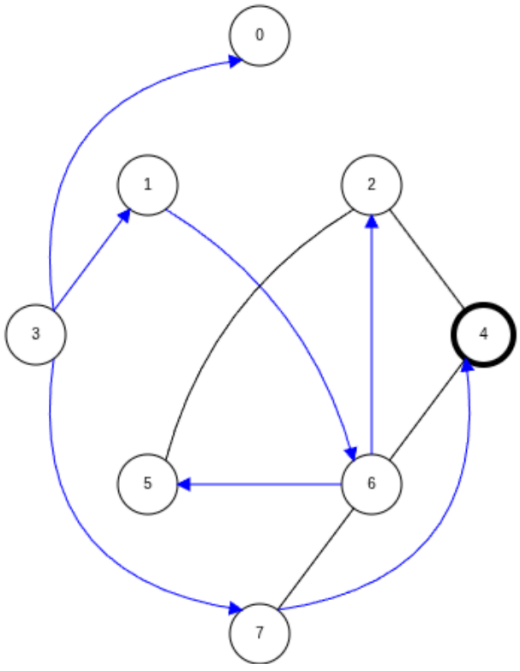
Parent Array

0	1	2	3	4	5	6	7
3	3	6	N	7	6	1	3

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

Step 7: dequeuing 4 and processing its neighbors

Traversal Sequence: 3 0 1 7 6 4



We deque vertex 4 and do nothing because we see that all its neighbors are already visited.

Queue: 2 5

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	T	T

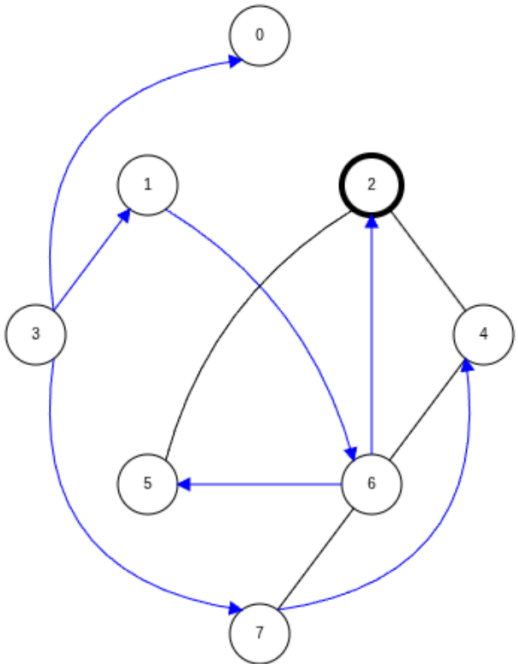
Parent Array

0	1	2	3	4	5	6	7
3	3	6	N	7	6	1	3

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

Step 8: dequeuing 2 and processing its neighbors

Traversal Sequence: 3 0 1 7 6 4 2



We deque vertex 2 and do nothing because we see that all its neighbors are already visited.

Queue: 5

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	T	T

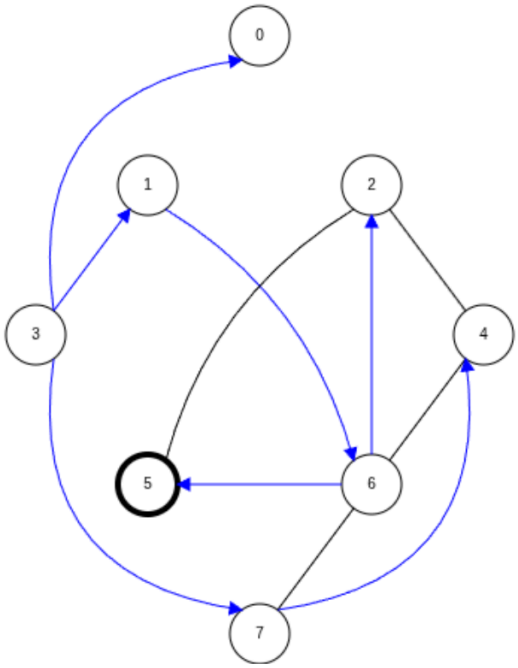
Parent Array

0	1	2	3	4	5	6	7
3	3	6	N	7	6	1	3

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

Step 9: dequeuing 5 and processing its neighbors

Traversal Sequence: 3 0 1 7 6 4 2 5



We deque vertex 5 and do nothing because we see that all its neighbors are already visited.

Queue:

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	T	T

Parent Array

0	1	2	3	4	5	6	7
3	3	6	N	7	6	1	3

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

This concludes our **BFS traversal** since our **Queue is empty**.
`BFS Traversal Sequence (starting at 3): 3 0 1 7 6 4 2 5`

NOTE: This process will traverse all the possible vertices from a specific starting point. If the traversal graph has a disconnected portion or its a directed graph with some vertices having no incoming edges. We need to repeat this process until all vertices have been visited. You can compare the simulation with the pseudocode to understand it better.

Reference: https://www.cs.usfca.edu/~galles/visualization/BFS.html

7.4.1.2 `DFS SIMULATION (using recursion) :`

We'll be simulating **DFS using 2 extra data structures**:

- **A Visited Array:** A boolean array to keep track of vertices which are visited
  - By default all the vertices will be unvisited (false)
  - As we visit each vertex, we'll make them true.
- **A Parent Array:** to keep track of the edges/paths the algorithm has discovered.
  - By default the values are going to be null.

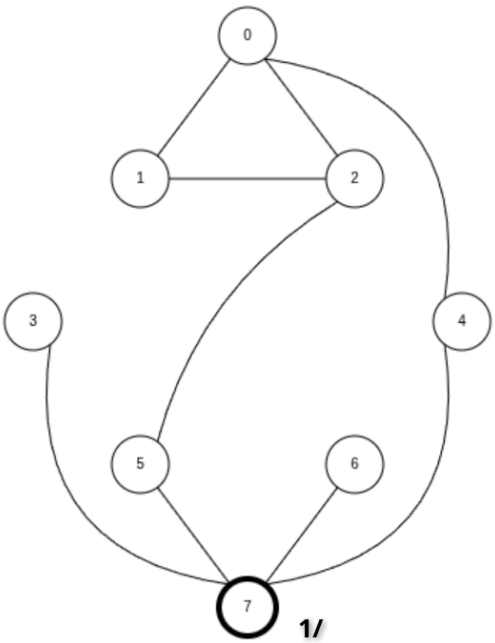
We'll keep track of starting & ending time of each vertex for DFS simulation

`DFS Traversal on an Undirected Unweighted Graph`

Step 1: Starting vertex is 7

`Traversal Sequence: 7`

Recursive Call Stack



The **starting time of 7 is 1**, the ending time will be shown beside the / (forward slash symbol) when vertex 7 has been processed.

Visited Array

0	1	2	3	4	5	6	7
F	F	F	F	F	F	F	**T**

Parent Array

0	1	2	3	4	5	6	7
N	N	N	N	N	N	N	N

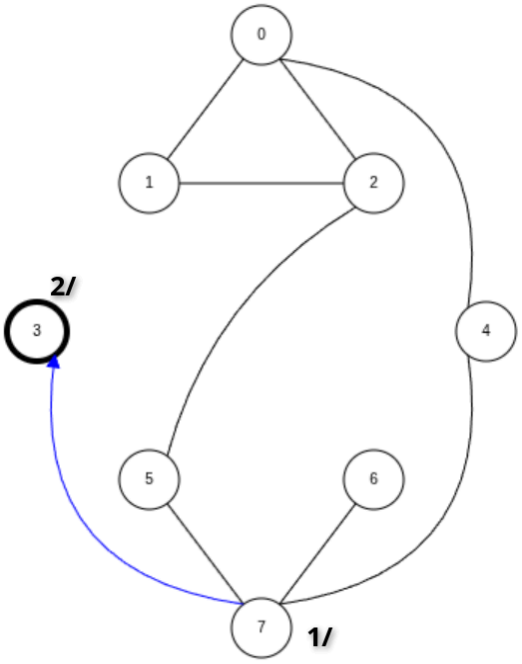
`DFS(7)`

`DFS(7) has been called.`

Step 2: visit vertex 7's neighbor 3

Recursive Call Stack

`Traversal Sequence: 7 3`



The **starting time of vertex 3** is 2.

Visited Array

0	1	2	3	4	5	6	7
F	F	F	**T**	F	F	F	**T**

Parent Array

0	1	2	3	4	5	6	7
N	N	N	**7**	N	N	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

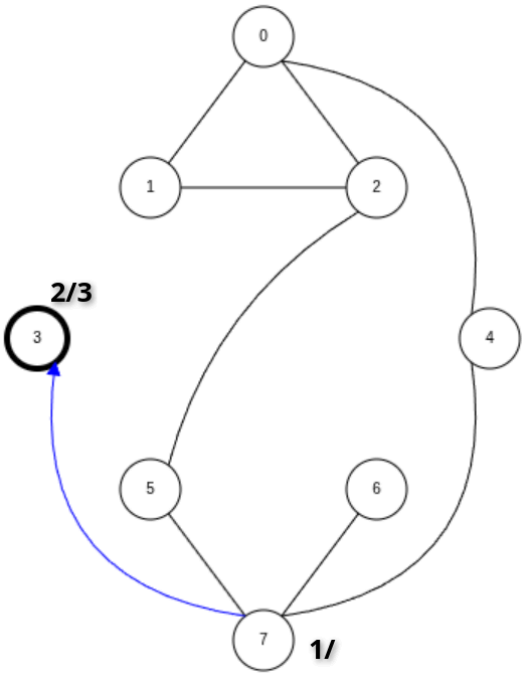
```
DFS(7)
 DFS(3)


DFS(3) has
been called
from inside
of DFS(7).
```

Step 3: since vertex 3 has no neighbor we've already processed 3

Recursive Call Stack

`Traversal Sequence: 7 3`



Since vertex 3 has been processed (meaning no unvisited neighbors left) so we can update the **ending time of vertex 3** as 3.

Visited Array

0	1	2	3	4	5	6	7
F	F	F	**T**	F	F	F	**T**

Parent Array

0	1	2	3	4	5	6	7
N	N	N	**7**	N	N	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

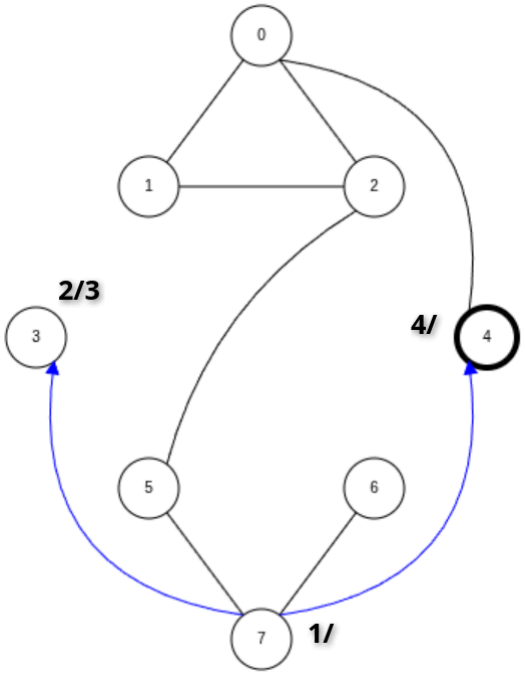
```
DFS(7)


DFS(3) call
has ended
and we
returned to
the previous
DFS(7)
function.
```

Step 4: We move back to vertex 7 and process next neighbor vertex 4

Traversal Sequence: 7 3 4

Recursive Call Stack



The **starting time of vertex 4** is 4.

Visited Array

0	1	2	3	4	5	6	7
F	F	F	**T**	**T**	F	F	**T**

Parent Array

0	1	2	3	4	5	6	7
N	N	N	**7**	**7**	N	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

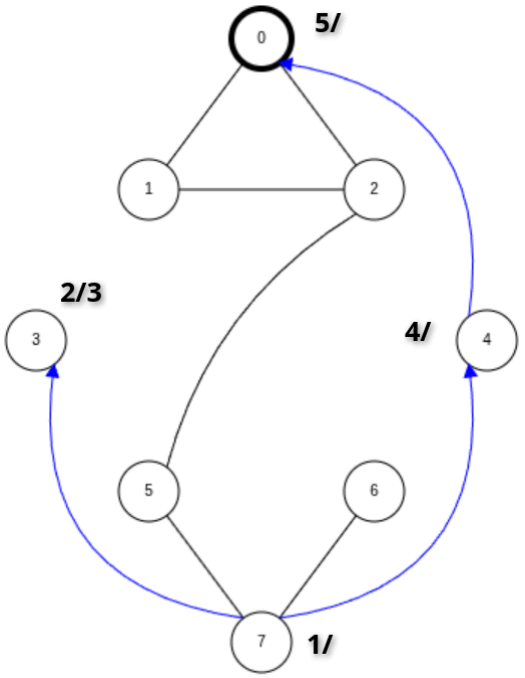
```
DFS(7)
 DFS(4)
```

`DFS(4) was called from inside of DFS(7).`

Step 5: We process vertex 0 (the neighbor of vertex 4)

Traversal Sequence: 7 3 4 0

Recursive Call Stack



The **starting time of vertex 0** is 5.

Visited Array

0	1	2	3	4	5	6	7
T	F	F	**T**	**T**	F	F	**T**

Parent Array

0	1	2	3	4	5	6	7
4	N	N	**7**	**7**	N	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

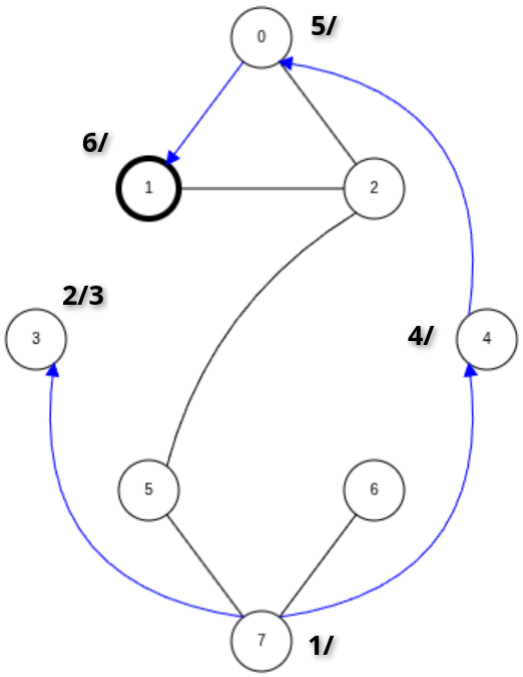
```
DFS(7)
 DFS(4)
  DFS(0)
```

`DFS(0) was called from inside of DFS(4).`

Step 6: We process vertex 1 (the neighbor of vertex 1)

Traversal Sequence: 7 3 4 0 1

Recursive Call Stack



The **starting time of vertex 1** is 6.

Visited Array

0	1	2	3	4	5	6	7
T	T	F	T	T	F	F	T

Parent Array

0	1	2	3	4	5	6	7
4	0	N	7	7	N	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

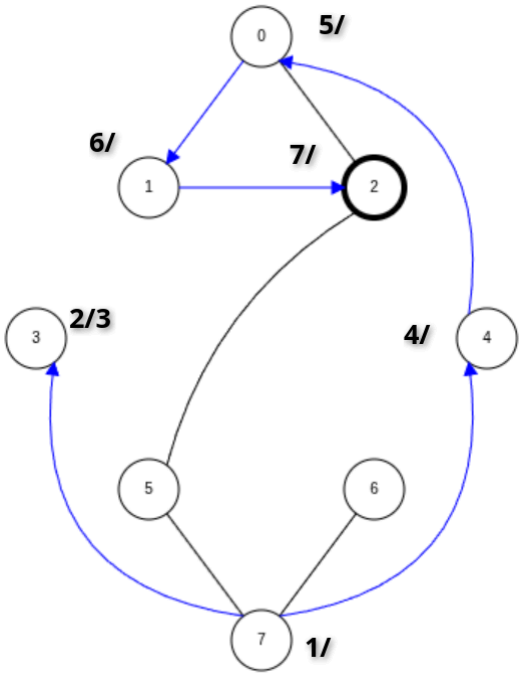
```
DFS(7)
 DFS(4)
  DFS(0)
   DFS(1)
```

DFS(1) was called from inside of DFS(0).

Step 7: We process vertex 2 (the neighbor of vertex 1)

Traversal Sequence: 7 3 4 0 1 2

Recursive Call Stack



The **starting time of vertex 2** is 7.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	F	F	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	N	N	N

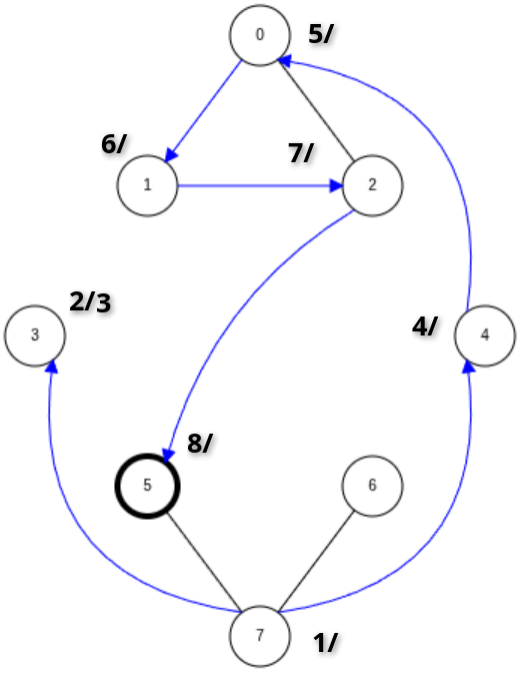
NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

```
DFS(7)
 DFS(4)
  DFS(0)
   DFS(1)
    DFS(2)
```

DFS(2) was called from inside of DFS(1).

Step 8: We process vertex 5 (the neighbor of vertex 2)

Traversal Sequence: 7 3 4 0 1 2 5



The **starting time of vertex 5** is 8.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	F	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

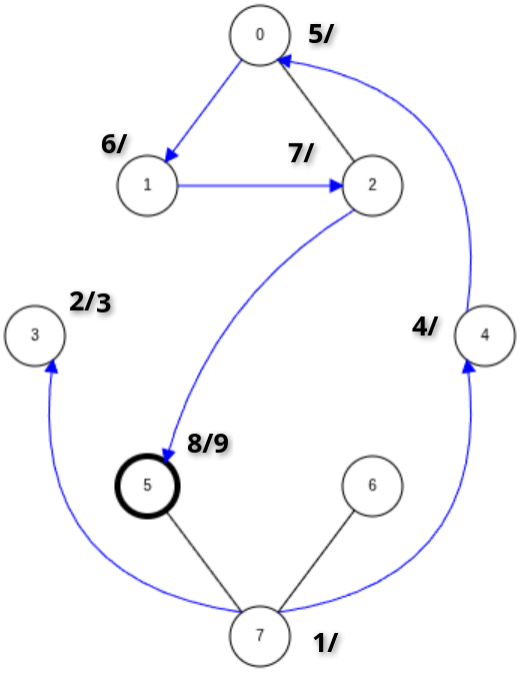
Recursive Call Stack

```
DFS(7)
 DFS(4)
  DFS(0)
   DFS(1)
    DFS(2)
     DFS(5)
```

DFS(5) was called from inside of DFS(2).

Step 9: ending vertex 5 and returning to vertex 2

Traversal Sequence: 7 3 4 0 1 2 5



Since **vertex 5** has been processed (meaning no unvisited neighbors left) so we can update the **ending time of vertex 5** as 9.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	F	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

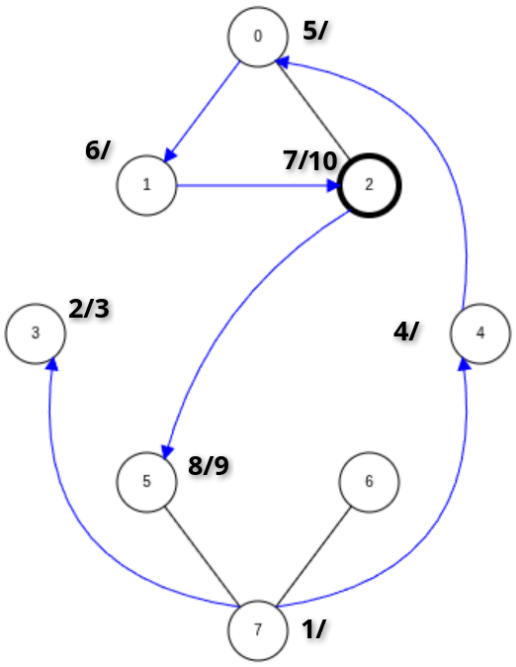
Recursive Call Stack

```
DFS(7)
 DFS(4)
  DFS(0)
   DFS(1)
    DFS(2)
```

DFS(5) call has ended and we returned to the previous DFS(2) function.

Step 10: ending vertex 2 and returning to vertex 1

Traversal Sequence: 7 3 4 0 1 2 5



Since **vertex 2** has been processed (meaning no unvisited neighbors left) so we can update the **ending time of vertex 2** as 10.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	F	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

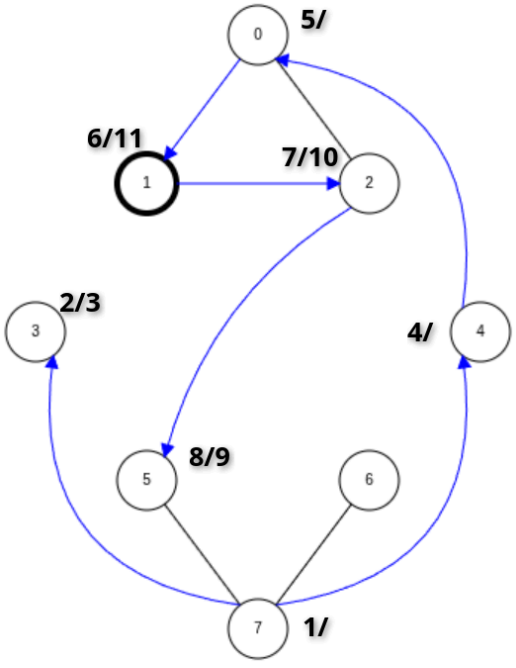
Recursive Call Stack

```
DFS(7)
 DFS(4)
  DFS(0)
   DFS(1)
```

DFS(2) call has ended and we returned to the previous DFS(1) function.

Step 11: ending vertex 1 and returning to vertex 0

Traversal Sequence: 7 3 4 0 1 2 5



Since **vertex 1** has been processed (meaning no unvisited neighbors left) so we can update the **ending time of vertex 1** as 11.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	F	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

Recursive Call Stack

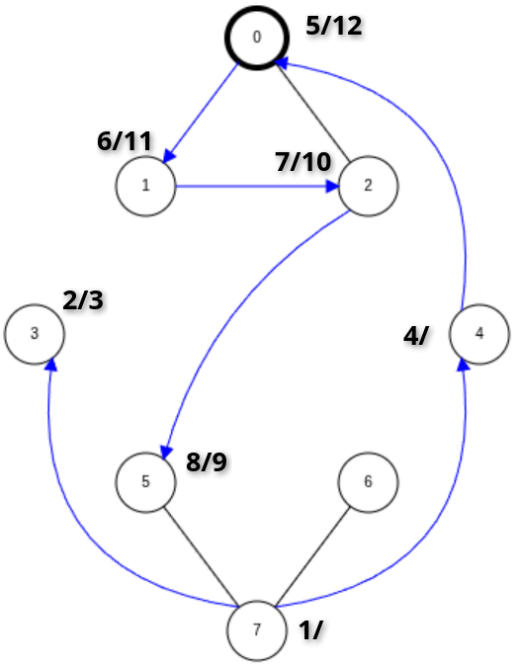
```
DFS(7)
 DFS(4)
  DFS(0)
```

DFS(1) call has ended and we returned to the previous DFS(0) function.

Step 12: ending vertex 0 and returning to vertex 4

Traversal Sequence: 7 3 4 0 1 2 5

Recursive Call Stack



Ignoring vertex 2 & 4 (neighbors of vertex 0) because they were already visited.
Thus, **vertex 0** has been processed (meaning no unvisited neighbors left) so we can update the **ending time of vertex 0** as 12.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	F	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	N	N

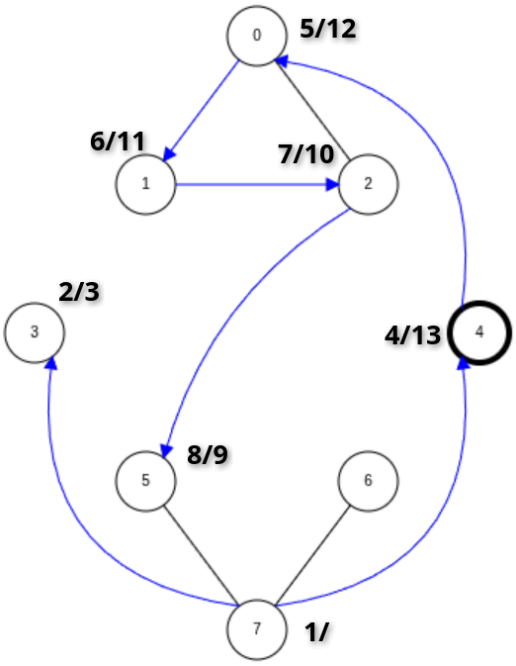
```
DFS(7)
 DFS(4)

DFS(0) call has ended and we returned to the previous DFS(4) function.
```

Step 13: ending vertex 4 and returning to vertex 7

Traversal Sequence: 7 3 4 0 1 2 5

Recursive Call Stack



Since **vertex 4** has been processed (meaning no unvisited neighbors left) so we can update the **ending time of vertex 4** as 13.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	F	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	N	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

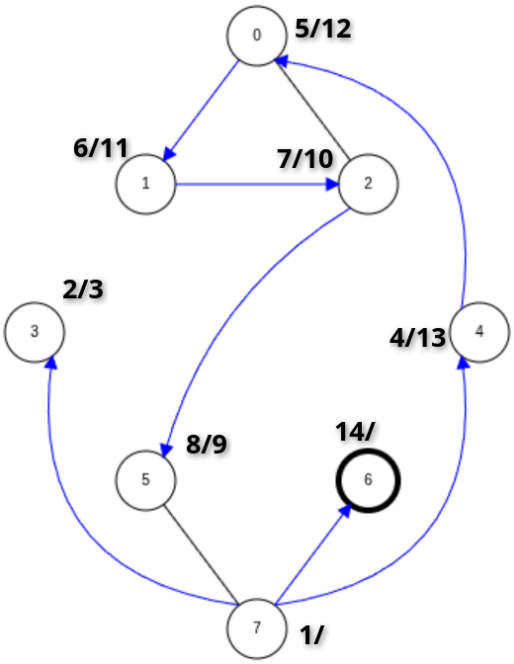
```
DFS(7)

DFS(4) call has ended and we returned to the previous DFS(7) function.
```

Step 14: processing vertex 6 (neighbor of vertex 7)

Traversal Sequence: 7 3 4 0 1 2 5 6

Recursive Call Stack



The **starting time of vertex 6** is 14.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	T	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	7	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

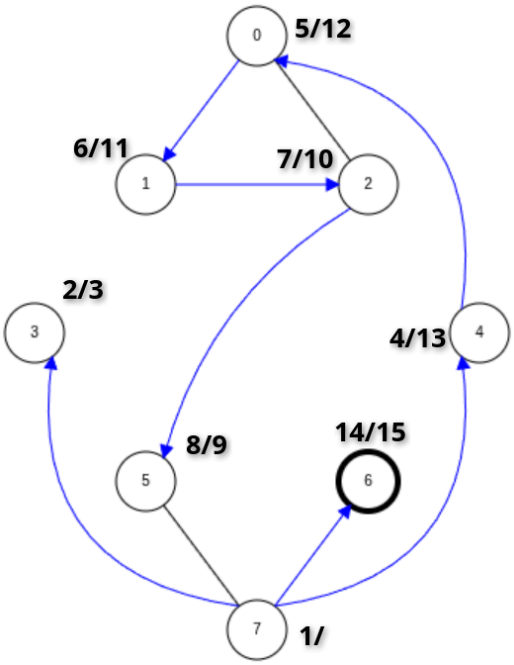
```
DFS(7)
 DFS(6)


DFS(6) was
called from
inside of
DFS(7).
```

Step 15: ending vertex 6 and returning to vertex 7

Traversal Sequence: 7 3 4 0 1 2 5 6

Recursive Call Stack



Since **vertex 6** has been processed (meaning no unvisited neighbors left) so we can update the **ending time of vertex 6** as 15.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	T	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	7	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

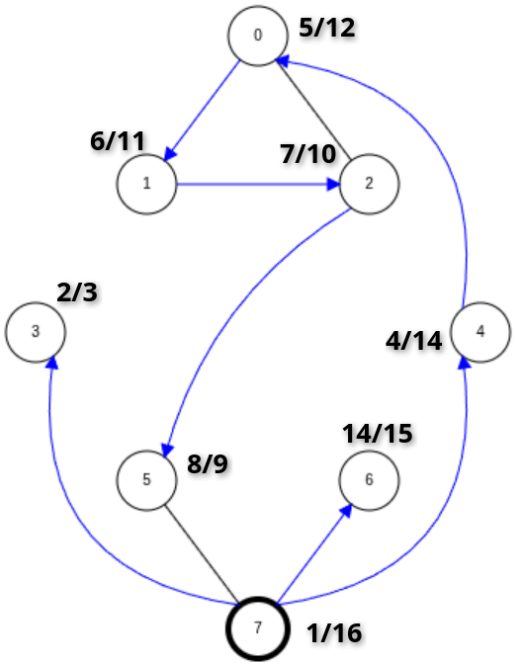
```
DFS(7)


DFS(6) call
has ended
and we
returned to
the previous
DFS(7)
function.
```

Step 16: ending vertex 7 & concluding the DFS traversal

Traversal Sequence: 7 3 4 0 1 2 5 6

Recursive Call Stack



Since **vertex 6** has been processed (meaning no unvisited neighbors left) so we can update the **ending time of vertex 6** as 15.

Visited Array

0	1	2	3	4	5	6	7
T	T	T	T	T	T	T	T

Parent Array

0	1	2	3	4	5	6	7
4	0	1	7	7	2	7	N

NOTE: The blue colored edges are NOT DIRECTED. It just means we've discovered that edge

DFS(7) has ended concluding our DFS traversal.

This concludes our **DFS traversal** since no more recursive calls are left.

DFS Traversal Sequence (starting at 7): 7 3 4 0 1 2 5 6

NOTE: This process will traverse all the possible vertices from a specific starting point. If the traversal graph has a disconnected portion or its a directed graph with some vertices having no incoming edges. We need to repeat this process until all vertices have been visited. You can compare the simulation with the pseudocode to understand it better.

Reference: https://www.cs.usfca.edu/~galles/visualization/DFS.html

You can try to simulate BFS/DFS on a Directed Graph on your own for practice

The algorithm for graph traversal visits each vertex only once and when visiting it, it has to evaluate all its edges. For an undirected graph, each edge is evaluated twice as the edge is considered from the both endpoints. Hence, if a graph has n vertices and m edges, the traversal itself takes n + 2m steps. However, in each step, checking whether the other endpoint of an edge is in the visited list takes n steps in the worst case. So the running time of the traversal algorithm is $O(n + 2m + n^2) = O(m + n^2)$. However, if we use a Boolean array instead of the visited list for tracking the already considered vertices, then checking if a vertex needs to be put in the store takes a constant time. Then the running time of traversal becomes O(n + m).